

LAPORAN PRATIKUM STRUKTUR DATA

Stuck



Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Wahyudi. DR, S.T.M.T

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

2026

KATA PENGANTAR

Puji dan syukur kita panjatkan kehadiran Allah yang Maha Esa karena telah memberikan nikmat sehat jasmani dan rohani sehingga saya bisa menyelesaikan laporan ini. Di dalam laporan ini, terdapat hasil praktikum, pembahasan serta kesimpulan yang saya susun secara sistematis. Saya sebagai penulis sekaligus penyusun laporan ini sangat berterimakasih kepada Bapak Wahyudi selaku dosen dalam mata kuliah praktikum Struktur Data Departemen Informatika, Universitas Andalas, yang telah menuntun dan mengajarkan saya mata kuliah ini.

Dalam penulisan laporan praktikum ini, saya mendapatkan suatu pelajaran baik serta mendapatkan pengetahuan tentang praktikum tersebut. Saya sebagai penulis dalam laporan ini masih dalam proses belajar, baik dalam tata penulisan laporan yang benar maupun dalam memahami penyajian laporan yang sesuai kaidah. Untuk itu saya berharap mendapatkan saran-saran yang membangun sehingga dapat meningkatkan pengetahuan saya dan dapat membuat isi laporan yang lebih baik, mendidik dan dapat dipertanggungjawabkan.

Padang, 2026

Shofwatul Hasanah Sarma

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan Pratikum	3
1.3 Manfaat Pratikum	4
BAB II.....	5
PEMBAHASAN	5
2.1 Pengertian Stuck.....	5
2.2 Alat dan Bahan.....	5
2.3 Langkah langkah	6
BAB III	24
KESIMPULAN.....	24
3.1 Ringkasan Hasil Pratikum	24
3.2 Saran Pengembangan.....	24
DAFTAR PUSTAKA	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman komputer yang digunakan untuk mengatur dan menyimpan data agar dapat diakses secara efisien. Salah satu struktur data yang sering digunakan adalah Stack. Stack merupakan struktur data yang bekerja dengan prinsip LIFO (Last In First Out), yaitu data yang terakhir masuk akan menjadi data pertama yang keluar.

Struktur data Stack banyak digunakan dalam berbagai aplikasi seperti sistem undo/redo, browser history, pemanggilan fungsi (recursion), serta evaluasi ekspresi **matematika**. Oleh karena itu, penting bagi mahasiswa untuk memahami konsep Stack melalui praktikum agar dapat mengimplementasikannya dalam pemrograman secara langsung.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep Stack, operasi-operasi dasar Stack seperti push, pop, peek, dan isEmpty, serta mampu mengimplementasikannya menggunakan bahasa pemrograman seperti Java.

1.2 Tujuan Pratikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar struktur data stack
2. Mengetahui prinsip kerja Stack (LIFO)
3. Mempelajari operasi-operasi dasar pada Stack
4. Mengimplementasikan Stack menggunakan bahasa pemrograman
5. Melatih kemampuan pemrograman dalam penggunaan struktur data Stack

1.3 Manfaat Pratikum

Adapun manfaat dari praktikum Stuck ini adalah sebagai berikut:

1. Mahasiswa dapat memahami konsep struktur data Stack secara lebih mendalam
2. Mahasiswa mampu mengimplementasikan Stack dalam program
3. Meningkatkan kemampuan logika pemrograman
4. Memudahkan mahasiswa memahami struktur data lainnya
5. Sebagai dasar dalam pengembangan aplikasi seperti browser, kalkulator, dan sistem undo

BAB II

PEMBAHASAN

2.1 Pengertian Stuck

Stack adalah struktur data linear yang menggunakan prinsip Last In First Out (LIFO), dimana elemen terakhir yang dimasukkan ke dalam Stack akan menjadi elemen pertama yang dikeluarkan.

Operasi dasar pada Stack meliputi:

1. Push
Operasi untuk menambahkan data ke dalam Stack
2. Pop
Operasi untuk menghapus data teratas dari Stack
3. Peek / Top
Operasi untuk melihat data paling atas tanpa menghapusnya
4. isEmpty
Untuk mengecek apakah Stack kosong
5. isFull (Jika menggunakan array)
Untuk mengecek apakah Stack penuh

Contoh penggunaan Stack dalam kehidupan sehari-hari adalah:

- Tumpukan buku
- Tumpukan piring
- Fitur undo pada Microsoft Word
- History browser

2.2 Alat dan Bahan

- a. Komputer / Laptop

Digunakan untuk mengaplikasikan aplikasi eclipse

- b. Aplikasi eclipse
- c. Paket data atau jaringan yang stabil

2.3 Langkah Langkah

#CLASS NilaiMaksimum_2511533015

```
package pekan3_2511533015;
import java.util.Stack;
public class NilaiMaksimum_2511533015 {
    public static int max_3015 (Stack <Integer> s_3015) {
        Stack<Integer> backup_3015 = new Stack <Integer>();
        int max_value_3015 = s_3015.pop ();
        backup_3015.push (max_value_3015);
        while (!s_3015.isEmpty()) {
            int next = s_3015.pop();
            backup_3015.push(next);
            max_value_3015 = Math.max (max_value_3015, next);
        }
        while (!backup_3015.isEmpty()) {
            s_3015.push (backup_3015.pop());
        }
        return max_value_3015;
    }
    public static void main (String[] args) {
        Stack <Integer> s_3015 = new Stack <Integer>();
        s_3015.push (70);
        s_3015.push (12);
        s_3015.push (20);
        System.out.println("isi stack " + s_3015);
        System.out.println("Stack Teratas" + s_3015.peek());
        System.out.println("Nilai maksimum" + max_3015 (s_3015));
    }
}
```

PENJELASAN

1. Package dan import

= package pekan3_2511533015;

import java.util.Stack;

- package pekan3_2511533015; → Menentukan nama package program
- import java.util.Stack; → Menggunakan class Stack dari Java

2. deklarasi class

= public class NilaiMaksimum_2511533015 {
Membuat class bernama NilaiMaksimum_2511533015
sebagai program utama.

3. Method max_3015

= public static int max_3015 (Stack <Integer> s_3015)
Method ini digunakan untuk mencari nilai terbesar dalam
Stack

penjelasan kode

= Stack<Integer> backup_3015 = new Stack <Integer>();

Membuat stack baru bernama backup_3015 untuk
menyimpan sementara data

= int max_3015 = s_3015.pop ();

Mengambil nilai pertama dari stack sebagai nilai
maksimum awal

= backup_3015.push (max_3015);

Memasukkan nilai pertama ke backup stack

perulangan mencari nilai maksimum

= while (!s_3015.isEmpty())

Loop selama stack belum kosong

= int next = s_3015.pop();

Ambil nilai berikutnya dari stack

= backup_3015.push(next);

Simpan nilai ke backup stack

= max_3015 = Math.max (max_3015, next);

Bandingkan nilai sebelumnya dengan nilai baru

Kemudian ambil nilai yang lebih besar

mengembalikan isi stack seperti semula

= while (!backup_3015.isEmpty()) {

 s_3015.push (backup_3015.pop());

}

Mengembalikan semua data dari backup stack ke stack utama

= return max_value_3015;

Mengembalikan nilai maksimum

4. Method m,ain

= public static void main (String[] args)

Program utama dijalankan dari sini

membuat stack

= Stack <Integer> s_3015 = new Stack <Integer>();

Membuat stack kosong bernama s_3015

menambahkan data

s_3015.push (70);

s_3015.push (12);

s_3015.push (20);

menambahkan nilai ke stack

Isi stack menjadi:

70

12

20 ← paling atas

menampilkan isi stack

System.out.println("isi stack " +s_3015);

Output:

isi stack [70, 12, 20]

menampilkan stack teratas

= System.out.println("Stack Teratas"+ s_3015.peek());

peek() → melihat nilai teratas

output:

Stack Teratas 20

menampilkan nilai maksimum

```
= System.out.println("Nilai maksimum"+ max_3015  
(s_3015));
```

Memanggil fungsi max_3015 untuk mencari nilai terbesar

Output:

Nilai maksimum 70

output program:

isi stack [70, 12, 20]

Stack Teratas 20

Nilai maksimum 70

#CLASS Siswa_3015

```
package pekan3_2511533015;  
import java.util.ArrayList;  
class Siswa_3015 {  
    String nama_3015;  
    int nim_3015;  
  
    public Siswa_3015 (String nama_3015, int nim_3015) {  
        this.nama_3015 = nama_3015;  
        this.nim_3015 = nim_3015;  
    }  
  
    @Override  
    public String toString () {  
        return "Nim: " + nim_3015 + ", Nama: " + nama_3015;  
    }  
}  
  
public class SiswaStack_2511533015 {  
    private ArrayList <Siswa_3015> stack_3015;  
  
    public SiswaStack_2511533015 () {  
        stack_3015 = new ArrayList <>();  
    }  
  
    public void push_3015 (Siswa_3015 mhs) {  
        stack_3015.add (mhs);  
    }  
  
    public Siswa_3015 pop() {  
        if (!isEmpty_3015()) {  
            return stack_3015.remove (stack_3015.size() - 1);  
        }  
        return null;  
    }  
}
```

```

public Siswa_3015 peek() {
    if (!isEmpty_3015()) {
        return stack_3015.get (stack_3015.size() - 1) ;
    }
    return null;
}

public boolean isEmpty_3015() {
    return stack_3015.isEmpty();
}

public void tampilkanSiswa () {
    for (int i = stack_3015.size () - 1; i >= 0; i--) {
        System.out.println (stack_3015.get(i));
    }
}

public static void main (String[] args) {
    SiswaStack_2511533015 studentStack = new
SiswaStack_2511533015 ();

    Siswa_3015 mhs1 = new Siswa_3015 ("Ali", 1);
    Siswa_3015 mhs2 = new Siswa_3015 ("Boby", 2);
    Siswa_3015 mhs3 = new Siswa_3015 ("Charles", 3);

    studentStack.push_3015(mhs1);
    studentStack.push_3015(mhs2);
    studentStack.push_3015(mhs3);

    System.out.println ("Siswa di dalam stack:");
    studentStack.tampilkanSiswa();

    System.out.println ("siswa teratas " + studentStack.peek ());
    System.out.println ("mengeluarkan siswa teratas dari stack: " +
studentStack.pop());
    System.out.println ("daftar siswa setelah di pop :");
    studentStack.tampilkanSiswa ();
}
}

```

PENJELASAN:

1. Class siswa_3015

```

= class Siswa_3015 {
    String nama_3015;
    int nim_3015;

```

Class ini digunakan untuk menyimpan data siswa:

- nama_3015 → Menyimpan nama siswa

- nim_3015 → Menyimpan NIM siswa

```
# constructor
= public Siswa_3015 (String nama_3015, int
nim_3015) {
    this.nama_3015 = nama_3015;
    this.nim_3015 = nim_3015;
}
```

Constructor digunakan untuk **mengisi data siswa** saat objek dibuat.

Contoh:

```
= Siswa_3015 mhs1 = new Siswa_3015 ("Ali", 1);
```

```
# method toString()
```

```
= @Override
```

```
public String toString () {
    return "Nim: " + nim_3015 + ", Nama: " +
nama_3015;
}
```

Method ini digunakan untuk menampilkan data siswa

Output contoh:

Nim: 1, Nama: Ali

2. class SiswaStack_2511533015

```
= private ArrayList <Siswa_3015> stack_3015;
```

Membuat Stack menggunakan ArrayList

```
# constructor Stack
```

```
= public SiswaStack_2511533015 () {
    stack_3015 = new ArrayList <>();
}
```

Membuat stack kosong

3. method push

```
= public void push_3015 (Siswa_3015 mhs) {
    stack_3015.add (mhs);
```

```
}
```

Digunakan untuk menambahkan siswa ke stack

Contoh:

```
push(Ali)
```

```
push(Boby)
```

```
push(Charles)
```

stack:

Ali

Boby

Charles ← teratas

4. method pop

```
= public Siswa_3015 pop() {  
    if (!isEmpty_3015()) {  
        return stack_3015.remove  
        (stack_3015.size() - 1);  
    }  
    return null;  
}
```

Digunakan untuk menghapus data paling atas

Contoh:

Charles keluar dari stack

5. method peek

```
public Siswa_3015 peek() {  
    if (!isEmpty_3015 ()) {  
        return stack_3015.get (stack_3015.size() - 1)  
    }  
    return null;  
}
```

Digunakan untuk melihat siswa paling atas tanpa menghapus

Contoh:

Charles ← siswa teratas

6. Method isEmpty

```
= public boolean isEmpty_3015() {  
    return stack_3015.isEmpty();  
}
```

Digunakan untuk mengecek apakah stack kosong

7. Method tampilaknSiswa

```
= public void tampilkanSiswa () {  
    for (int i = stack_3015.size () - 1; i >= 0; i--) {  
        System.out.println (stack_3015.get(i));  
    }  
}
```

Digunakan untuk menampilkan isi stack dari atas ke bawah

8. Method main

membuat objek stack

SiswaStack_2511533015 studentStack = new

SiswaStack_2511533015 ();

membuat data siswa

Siswa_3015 mhs1 = new Siswa_3015 ("Ali", 1);

Siswa_3015 mhs2 = new Siswa_3015 ("Boby", 2);

Siswa_3015 mhs3 = new Siswa_3015 ("Charles", 3);

push data ke stack

studentStack.push_3015(mhs1);

studentStack.push_3015(mhs2);

studentStack.push_3015(mhs3);

stack:

Ali

Boby

Charles ← teratas

```
# menampilkan stack  
= studentStack.tampilkanSiswa();
```

Output:

Nim: 3, Nama: Charles

Nim: 2, Nama: Bobby

Nim: 1, Nama: Ali

```
# peek  
= System.out.println ("siswa teratas " +  
studentStack.peek ());
```

Output:

siswa teratas Nim: 3, Nama: Charles

```
# pop  
= System.out.println ("mengeluarkan siswa teratas dari  
stack: " + studentStack.pop());
```

Output:

mengeluarkan siswa teratas dari stack: Nim: 3, Nama:
Charles

```
#menampilakn stuck setelah pop  
= studentStack.tampilkanSiswa ();
```

Output:

Nim: 2, Nama: Bobby

Nim: 1, Nama: Ali

```
#CLASS stackArray_2511533015
```

```
package pekan3_2511533015;  
  
public class stackArray_2511533015 {  
    static final int MAX_3015 = 1000;  
    int top_3015;  
    int a [] = new int [MAX_3015];  
    boolean isEmpty_3015 ()  
    {  
        return (top_3015 < 0);  
    }  
    stackArray_2511533015 ()
```

```

    {
        top_3015 = -1;
    }
    boolean push_3015 (int x)
    {
        if (top_3015 >= (MAX_3015 - 1)) {
            System.out.println ("Stack Overflow");
            return false;
        } else {
            a[++top_3015] = x;
            System.out.println (x + " dimasukkan dalam
stack");
            return true;
        }
    }
    int pop_3015 ()
    {
        if (top_3015 < 0 ) {
            System.out.println ("Stack Underflow");
            return 0;
        } else {
            int x = a[top_3015--];
            return x;
        }
    }
    int peek_3015()
    {
        if (top_3015 < 0) {
            System.out.println ("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top_3015];
            return x;
        }
    }
    void print () {
        for (int i = top_3015; i > -1; i--) {
            System.out.print(" " + a[i]);
        }
    }
}

```

PENJELASAN

1. Deklarasi class

= public class stackArray_2511533015 {
Membuat class bernama stackArray_2511533015

2. Deklarasi variable


```
= static final int MAX_3015 = 1000;  
int top_3015;  
int a [] = new int [MAX_3015];
```

Penjelasan:

- MAX_3015 = 1000 → kapasitas maksimum stack
- top_3015 → penunjuk posisi atas stack
- a[] → array untuk menyimpan data stack

3. method isEmpty

```
= boolean isEmpty_3015 ()  
{  
    return (top_3015 < 0);  
}
```

Digunakan untuk mengecek apakah stack kosong

Jika top = -1 maka stack kosong

4. constructor

```
= stackArray_2511533015 ()  
{  
    top_3015 = -1;  
}
```

Constructor digunakan untuk:

Mengatur stack awal kosong dengan top = -1

5. method push

```
= boolean push_3015 (int x)
```

Digunakan untuk menambahkan data ke stack

```
= if (top_3015 >= (MAX_3015 - 1))
```

Jika stack penuh maka:

```
= System.out.println ("Stack Overflow");
```

Jika tidak penuh:

```
a [++top_3015] = x;
```

Menambahkan data ke stack

Contoh:

Push 10

Push 20

Push 30

Stack:

30 ← Top

20

10

6. method pop

= int pop_3015 ()

Digunakan untuk menghapus data paling atas

Jika stack kosong:

System.out.println ("Stack Underflow");

Jika tidak kosong:

int x= a [top_3015--];

Mengambil data teratas lalu mengurangi top

Contoh:

Stack:

30 ← Top

20

10

Pop → 30 keluar

Stack menjadi:

20 ← Top

10

7. method peek

= int peek_3015()

Digunakan untuk melihat data teratas tanpa menghapus

= int x = a [top_3015];

Contoh:

Stack:

30 ← Top

20

10

Peek → 30

8. method print

= void print ()

Digunakan untuk menampilkan isi stack

for (int i = top_3015; i>=0; i--)

Menampilkan dari atas ke bawah

Contoh Output:

30 20 10

CLASS stackArrayDriver_2511533015

```
package pekan3_2511533015;

public class stackArrayDriver_2511533015 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        stackArray_2511533015 s= new stackArray_2511533015 ();
        s.push_3015 (10);
        s.push_3015(20);
        s.push_3015(30);
        System.out.println (s.pop_3015() + " dikeluarkan dari stack");
        System.out.println ("Elemen teratas adalah : " + s.peek_3015 ());
        System.out.print("Elemen pata stack : ");
        s.print ();

    }

}
```

PENJELASAN

1. deklarasi class

= public class stackArrayDriver_2511533015 {

Class ini adalah **program utama** yang memiliki method main().

2. Method main

= public static void main(String[] args) {

Method main() adalah tempat program dijalankan.

3. Membuat objek stack

```
= stackArray_2511533015 s= new stackArray_2511533015  
();
```

Membuat objek stack bernama s dari class
stackArray_2511533015.

Stack awal:

Kosong

4. Push data ke stack

```
s.push_3015 (10);  
s.push_3015(20);  
s.push_3015(30);
```

Menambahkan data ke stack:

Push 10

Push 20

Push 30

Stack menjadi:

30 ← Top

20

10

5. Pop data

```
System.out.println (s.pop_3015() + " dikeluarkan dari  
stack");
```

Menghapus data teratas

Data yang keluar:

30

Stack menjadi:

20 ← Top

20 ← Top

10

Output

30 dikeluarkan dari stack

6. Peek

```
= System.out.println ("Elemen teratas adalah : " +  
s.peek_3015 ());
```

Melihat data paling atas tanpa menghapus

Output:

Elemen teratas adalah : 20

7. Menampilkan isi stack

```
= System.out.print("Elemen pada stack : ");
```

```
s.print ();
```

Menampilkan isi stack dari atas ke bawah

Output:

Elemen pada stack : 20 10

#CLASS StackPostfix_2511533015

```
1 package pekan3_2511533015;  
2 import java.util.Scanner;  
3  
4 public class StackPostfix_2511533015 {  
5     public static int postfixEvaluate(String expression) {  
6         Stack<Integer> s_3015 = new Stack<Integer>();  
7         Scanner input_3015 = new Scanner(expression);  
8         while (input_3015.hasNext()) {  
9             if (input_3015.hasNextInt()) { // an operand (integer)  
10                s_3015.push(input_3015.nextInt());  
11            } else {  
12                String operator_3015 = input_3015.next();  
13                int operand2_3015 = s_3015.pop();  
14                int operand1_3015 = s_3015.pop();  
15                if (operator_3015.equals("+")) {  
16                    s_3015.push(operand1_3015 + operand2_3015);  
17                } else if (operator_3015.equals("-")) {  
18                    s_3015.push(operand1_3015 - operand2_3015);  
19                } else if (operator_3015.equals("*")) {  
20                    s_3015.push(operand1_3015 * operand2_3015);  
21                } else {  
22                    s_3015.push(operand1_3015 / operand2_3015);  
23                }  
24            }  
25        }  
26        input_3015.close();  
27        return s_3015.pop();  
28    }  
29    public static void main (String[] args) {  
30        System.out.println ("hasil postfix= " + postfixEvaluate ("5 2 4 * + 7 - "));  
31    }  
32 }  
3 }
```

PENJELASAN

1. IMPORT LIBRARY

```
= import java.util.Scanner;
```

```
import java.util.Stack;
```

- Scanner → Membaca string ekspresi postfix
- Stack → Menyimpan operand sementara

2. Method postfixEvaluate
= public static int postfixEvaluate(String expression)
Method ini digunakan untuk menghitung nilai postfix
3. Membuat stack
= Stack <Integer> s_3015 = new Stack <Integer>();
Membuat stack untuk menyimpan angka
4. Membaca ekspresi
= Scanner input_3015 = new Scanner (expression);
Membaca ekspresi postfix:
"5 2 4 * + 7 -"
5. Perulangan membaca ekspresi
= while (input_3015.hasNext ())
Loop membaca setiap angka atau operator
6. Jika operand (angka)
= if (input_3015.hasNextInt ())
Jika angka:
= s_3015.push (input_3015.nextInt ());
Masukkan angka ke stack
Contoh:
Push 5
Push 2
Push 4
Stack:
5
2
4
7. Jika operator
= String operator_3015 = input_3015.next ();
Mengambil operator
8. Mengambil dua operand
= int operand2_3015 = s_3015.pop();
int operand1_3015 = s_3015.pop();
Ambil dua angka dari stack

9. Operasi matematika

penjumlahan

if (operator_3015.equals("+"))

pengurangan

else if (operator_3015.equals("-"))

perkalian

else if (operator_3015.equals("*"))

pembagian

Else

10. Contoh proses perhitungan

Ekspresi:

5 2 4 * + 7 -

Langkah:

Push 5

Push 2

Push 4

Stack:

5 2 4

operator *

$2 \times 4 = 8$

stack:

5 8

#operator +

$5 + 8 = 13$

Stack:

13

Push 7

stack:

13 7

Operator -

$13 - 7 = 6$

Stack:

6

11. Mengembalikan hasil

= return s_3015.pop ();

Mengambil hasil akhir dari stack

12. Method main

= public static void main (String[] args)

Menjalankan program

13. Menampilkan hasil

= System.out.println ("hasil postfix= " + postfixEvaluate
("5 2 4 * + 7 - "));

Output:

hasil postfix= 6

BAB III

KESIMPULAN

1.1 Ringkasan Hasil Pratikum

Pada praktikum struktur data Stack, mahasiswa mempelajari konsep dasar Stack dan cara implementasinya menggunakan bahasa pemrograman Java. Mahasiswa membuat program yang dapat melakukan operasi dasar Stack seperti push, pop, peek, dan pengecekan kondisi Stack.

Program yang dibuat biasanya menggunakan:

- Array
- ArrayList
- Stack bawaan Java

Melalui praktikum ini mahasiswa dapat memahami bagaimana data disimpan dan dikelola menggunakan struktur Stack serta memahami penerapannya dalam aplikasi nyata seperti browser history dan sistem undo.

1.2 Saran Pengembangan

Agar praktikum struktur data Stack dapat dikembangkan lebih lanjut, maka dapat dilakukan beberapa hal berikut:

1. Mengembangkan Stack menggunakan GUI (Graphical User Interface)
2. Mengimplementasikan Stack pada aplikasi nyata seperti browser sederhana
3. Menggunakan Stack untuk evaluasi ekspresi matematika
4. Mengembangkan Stack menggunakan Linked List
5. Membandingkan performa Stack menggunakan Array dan Linked List

DAFTAR PUSTAKA

1. Data Structures and Algorithms in Java — Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser. Wiley, 2014.
2. Introduction to Algorithms — Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. MIT Press, 2009.
3. Struktur Data Menggunakan Java — Abdul Kadir. Andi Publisher, 2018.
4. Fundamentals of Data Structures — Ellis Horowitz dan Sartaj Sahni. Computer Science Press

Tugas Pekan 3

Jawaban

1. KELAS pertama = ADT (Website)

```
2. package tugaspekan3_2511533015;
3.
4. public class Website_2511533015 {
5.
6.     // Atribut
7.     private String judulWebsite_3015;
8.     private String urlWebsite_3015;
9.
10.    // Constructor
11.    public Website_2511533015(String judulWebsite, String urlWebsite) {
12.        this.judulWebsite_3015 = judulWebsite;
13.        this.urlWebsite_3015 = urlWebsite;
14.    }
15.
16.    // Getter
17.    public String getJudulWebsite_3015() {
18.        return judulWebsite_3015;
19.    }
20.
21.    public String getUrlWebsite_3015() {
22.        return urlWebsite_3015;
23.    }
24.
25.    // Setter
26.    public void setJudulWebsite_3015(String judulWebsite) {
27.        this.judulWebsite_3015 = judulWebsite;
28.    }
29.
30.    public void setUrlWebsite_3015(String urlWebsite) {
31.        this.urlWebsite_3015 = urlWebsite;
32.    }
33. }
```

Penjelasan:

1. Package

```
package tugaspekan3_2511533015;
```

Menentukan bahwa class ini berada dalam package tugaspekan3_2511533015

2. Deklarasi class

```
public class Website_2511533015 {
```

Membuat class bernama Website_2511533015

Class ini digunakan sebagai tipe data objek website

3. Atribut

```
= private String judulWebsite_3015;  
private String urlWebsite_3015;
```

Atribut yang digunakan:

- judulWebsite_3015 → menyimpan nama website
- urlWebsite_3015 → menyimpan alamat website

Keyword private digunakan agar data tidak bisa diakses langsung dari luar class (Encapsulation)

4. constructor

```
= public Website_2511533015(String judulWebsite, String urlWebsite) {  
    this.judulWebsite_3015 = judulWebsite;  
    this.urlWebsite_3015 = urlWebsite;  
}
```

Constructor digunakan untuk mengisi data website saat objek dibuat

Contoh penggunaan:

```
= Website_2511533015 web1 = new Website_2511533015("Google",  
"www.google.com");
```

5. getter

Getter digunakan untuk mengambil data

Getter Judul Website

```
public String getJudulWebsite_3015() {  
    return judulWebsite_3015;  
}
```

Digunakan untuk mengambil judul website

getter URL website

```
= public String getUrlWebsite_3015() {  
    return urlWebsite_3015;  
}
```

Digunakan untuk mengambil URL website

6. setter

Setter digunakan untuk mengubah data

Setter judul website:

```
= public void setJudulWebsite_3015(String judulWebsite) {  
    this.judulWebsite_3015 = judulWebsite;  
}
```

Digunakan untuk mengubah judul website

setter URL Website

```
= public void setUrlWebsite_3015(String urlWebsite) {  
    this.urlWebsite_3015 = urlWebsite;  
}
```

Digunakan untuk mengubah URL website

Contoh penggunaan:

```
Website_2511533015 web = new Website_2511533015("Google",  
"www.google.com");
```

```
System.out.println(web.getJudulWebsite_3015());
```

```
System.out.println(web.getUrlWebsite_3015());
```

Output:

Google

www.google.com

2. KELAS Kedua = Driver (playlist)

```
3. package tugaspekan3_2511533015;  
4. import java.util.Scanner;  
5. import java.util.Stack;  
6. public class Browser_2511533015 {  
7.  
8.     public static void main(String[] args) {  
9.         // TODO Auto-generated method stub  
10.        Scanner input_3015 = new Scanner(System.in);  
11.        Stack<Website_2511533015> history_3015 = new Stack<>();  
12.  
13.        int pilihan;  
14.
```

```

15.     do {
16.         System.out.println("\n=== Browser History NIM: 2511533015 ===");
17.         System.out.println("1. Kunjungi Website (Push)");
18.         System.out.println("2. Tombol Back (Pop)");
19.         System.out.println("3. Lihat Halaman Aktif (Peek)");
20.         System.out.println("4. Cek Status History");
21.         System.out.println("5. Keluar");
22.         System.out.print("Pilihan: ");
23.         pilihan = input_3015.nextInt();
24.         input_3015.nextLine();
25.
26.         switch (pilihan) {
27.
28.             case 1:
29.                 System.out.print("Masukkan Judul: ");
30.                 String judul_3015 = input_3015.nextLine();
31.
32.                 System.out.print("Masukkan URL: ");
33.                 String url_3015 = input_3015.nextLine();
34.
35.                 Website_2511533015 websiteBaru =
36.                     new Website_2511533015(judul_3015, url_3015);
37.
38.                 history_3015.push(websiteBaru);
39.                 System.out.println("Berhasil mengunjungi halaman!");
40.                 break;
41.
42.             case 2:
43.                 if (!history_3015.isEmpty()) {
44.                     Website_2511533015 hapus = history_3015.pop();
45.                     System.out.println("Kembali dari: "
46.                         + hapus.getJudulWebsite_3015());
47.                 } else {
48.                     System.out.println("History kosong!");
49.                 }
50.                 break;
51.
52.             case 3:
53.                 if (!history_3015.isEmpty()) {
54.                     Website_2511533015 aktif = history_3015.peek();
55.                     System.out.println("Halaman aktif:");
56.                     System.out.println("Judul : "
57.                         + aktif.getJudulWebsite_3015());
58.                     System.out.println("URL : "
59.                         + aktif.getUrlWebsite_3015());
60.                 } else {
61.                     System.out.println("Tidak ada halaman aktif!");
62.                 }
63.                 break;
64.
65.             case 4:

```

```

66.         if (history_3015.isEmpty()) {
67.             System.out.println("History kosong");
68.         } else {
69.             System.out.println("Jumlah history: "
70.                 + history_3015.size());
71.         }
72.         break;
73.
74.     case 5:
75.         System.out.println("Terima kasih!");
76.         break;
77.
78.     default:
79.         System.out.println("Pilihan tidak valid!");
80.     }
81.
82. } while (pilihan != 5);
83.
84. input_3015.close();
85.
86. }
87.
88. }

```

PENJELASAN

1. Import library

= import java.util.Scanner;

import java.util.Stack;

- Scanner → untuk input dari keyboard
- Stack → untuk menyimpan history website

2. Method main

= public static void main(String[] args)

Method utama untuk menjalankan program.

3. Membuat scanner dan stack

= Scanner input_3015 = new Scanner(System.in);

Stack<Website_2511533015> history_3015 = new Stack<>();

- input_3015 → membaca input pengguna

- history_3015 → stack untuk menyimpan history website

4.menu program

```
System.out.println("1. Kunjungi Website (Push)");
```

```
System.out.println("2. Tombol Back (Pop)");
```

```
System.out.println("3. Lihat Halaman Aktif (Peek)");
```

```
System.out.println("4. Cek Status History");
```

```
System.out.println("5. Keluar");
```

Menu pilihan:

1. Push

2. Pop

3. Peek

4. Cek history

5. Keluar

5.perulangan menu

```
= do {
```

Program akan terus berjalan sampai pengguna memilih **5 (Keluar)**

6.case 1-Push

```
= System.out.print("Masukkan Judul: ");
```

```
String judul_3015 = input_3015.nextLine();
```

Input judul website

```
System.out.print("Masukkan URL: ");
```

```
String url_3015= input_3015.nextLine();
```

Input URL website

membuat objek website

= Website_2511533015 websiteBaru =

new Website_2511533015(judul_3015, url_3015);

Membuat objek website baru

push ke stack

= history_3015.push(websiteBaru);

Menambahkan ke history browser

Contoh:

Masuk:

- Google
- YouTube
- Instagram

Stack:

Instagram ← aktif

YouTube

Google

7.case 2-pop

= Website_2511533015 hapus = history_3015.pop();

Menghapus halaman terakhir

Contoh

Instagram keluar

Stack menjadi:

YouTube ← aktif

Google

8.case 3-peek

```
= Website_2511533015 aktif = history_3015.peek();
```

Melihat halaman aktif tanpa menghapus

Output:

Halaman aktif:

Judul : YouTube

URL : www.youtube.com

9.case 4-cek status history

```
= history_3015.size()
```

Menampilkan jumlah history

Contoh:

Jumlah history: 2

10.case 5-keluar

```
= System.out.println("Terima kasih!");
```

Program berhenti

11. perulangan berhenti

```
= } while (pilihan != 5);
```

Program berhenti jika memilih 5

OUTPUT

